语义分析

在完成语义分析之后，我们已经画出了 AST 语法树，在这个基础上我们进一步进行对这个语义分析，主要包括两个方面的内容：

1. 建立符号表
2. 错误检测

其实错误检测也是在建立符号表的过程中同步进行的

语义分析总体结构

```
semantic
├── ErrorController.java
├── ErrorToken.java
├── SemanticAnalyse.java
└── symbolTable
    └── symbol
        ├── FuncSymbol.java
        ├── StackSymbolTable.java
        ├── Symbol.java
        ├── SymbolTable.java
        ├── SymbolType.java
        └── SymbolTypeMap.java
```

这里 `FuncSymbol` 类是 `Symbol` 的子类，`SymbolTable` 是总的符号表，`StackSymbolTable` 负责管理不同层级的符号表，`SymbolType` 是符号类型的枚举类，`SymbolTypeMap` 是通过字符串得到对应符号类型的一个辅助类。语义分析的主逻辑在 `SemanticAnalyse` 中，剩余的两个和 `Error` 相关的类负责记录语义分析中检测的错误。

符号表的建立

语义分析的第一步是建立符号表，从上次完成的语法分析得到的最后的AST语法树开始构建符号表，对于一个符号表来说，一方面需要能够在声明一个变量的时候将符号保存到表中，同时在后续调用对应的变量或者函数的时候也能够通过符号表快速找到这个符号。对于实验的源代码来说，由于存在符号的分层机制，所以可以在不同的层次来声明同一个名称的变量，就像下面这种情况：

```
int a = 1;
{
    int a = 2;
}
```

这样是合理的，在下面这个 block 中使用的 a 应该优先从当前这个作用域开始查找，找不到再从外层作用域一个一个找。那我们建立的符号表也要能够具有这个功能。

符号 Symbol 类

先从这个符号表中的符号类开始说明，通过上面对于作用域的说明，我们的每一个符号类都至少有以下这些属性：

1. 符号名 `symbolName`

2. 符号种类 `symbolType`
3. 符号所在作用域 `symbolLevel`

目前就这些就足够了，但是后续还是要增加一些属性。这里的 符号种类 `symbolType` 实际上是自己定义的一个枚举类 `SymbolType` ，这个枚举类定义了符号的所有类型：

```
public enum SymbolType {
    Int, ConstInt, StaticInt, IntArray, ConstIntArray, StaticIntArray,
    Char, ConstChar, StaticChar, CharArray, ConstCharArray, StaticCharArray,
    VoidFunc, IntFunc, CharFunc,
}
```

当然这样的符号定义远远不能满足我们的需求，尤其是当我们要处理一个函数的符号的时候：

```
int f(int a, int b, int c) {
    ……
}
```

当我们想要记录这个函数对应的符号的时候我们会发现上面的三个属性完全不足以记录函数的信息，我们还需要记录这个函数的

1. 返回类型
2. 参数列表

这里可以很容易想到可以以之前的 Symbol 作为基类，生成一个子类 `FuncSymbol` ，在原来的基础上增加了上述的两个属性。

```
public class FuncSymbol extends Symbol {
    List<SymbolType> list;
    private SemanticAnalyse.ExprType returnType;
    public FuncSymbol(String symbolName, SymbolType symbolType, int symbolLevel,
int scopeLevel, List<SymbolType> list, SemanticAnalyse.ExprType returnType) {
        super(symbolName, symbolType, symbolLevel, scopeLevel);
        this.list = list;
    }
    public List<SymbolType> getList() { return list; }
}
```

这里的返回类型 `ExprType` 写在 `SemanticAnalyse` 中，这样设计没有什么别的原因，就是觉得类写的太多，就放在这里面没有提出来了…… 写这个类的主要原因在于可以观察这个函数定义的文法：

```
bType
    : 'int'
    | 'char'
    ;

funcDef
    : funcType IDENT '(' funcFParams? ')' block
    ;

funcType
    : 'void'
    | 'int'
```

```
    | 'char'
    ;

funcFParams
    : funcFParam (',' funcFParam)*
    ;

funcFParam
    : bType IDENT ('[' ']')?
    ;
```

从这里可以看到，对于一个函数的声明，函数的返回类型只有三种：`int`，`char`，`void`，而传入的参数类型也只有`int`，`char`，以及对应的数组类型。所以这边就写了一个`ExprType`类：

```
    public enum ExprType {
        INT,
        CHAR,
        INT_ARRAY,
        CHAR_ARRAY,
        VOID,
        ERROR
    }
```

这个类的作用很大，不仅现在用于记录函数对应的信息，在后续表达式计算的过程中还会大量使用。

`SymbolTable` 符号表

完成了对于`Symbol`的定义之后，接下来就是对这个符号表类型的设计了。前面提到了，这个`SymbolTable`是一个总的符号设计表，实际上的符号设计表是分层进行存储的，对于每一个作用域定义对应的符号表。按照这个思路，其实这个`SymbolTable`的设计就很简单：

```
public class SymbolTable {
    private final HashMap<Integer, StackSymbolTable> symbolTables = new HashMap<>
();  // 每个分析器独立的符号表

    public HashMap<Integer, StackSymbolTable> getSymbolTables() {
        return symbolTables;
    }

    public SymbolTable() {
        symbolTables.put(0, new StackSymbolTable());
    }

    public boolean addSymbol(Symbol symbol) {
        // 如果当前层为空，新建一个 symbolTables
        StackSymbolTable stackSymbolTable = symbolTables.computeIfAbsent(
                symbol.getScopeLevel(),
                id -> new StackSymbolTable()
        );
        // 如果返回 true 表示成功插入一个空位置，返回 false 表示重定义
        return
stackSymbolTable.getSymbolStack().putIfAbsent(symbol.getSymbolName(), symbol) ==
null;
    }
```

```
    /* 在对应的层中查找符号 */
    public Symbol findInScope(String symbolName, int scopeLevel) {
        StackSymbolTable stackSymbolTable = symbolTables.get(scopeLevel);
        if(stackSymbolTable == null) {
            return null;
        }
        return stackSymbolTable.getSymbolStack().get(symbolName);
    }

    public void printSymbolTable(BufferedWriter fwOut) throws IOException {
        List<Integer> scopeLevels = new ArrayList<>(symbolTables.keySet());
        Collections.sort(scopeLevels);

        for(Integer scopeLevel : scopeLevels) {
            StackSymbolTable stackSymbolTable = symbolTables.get(scopeLevel);
            for(Symbol symbol : stackSymbolTable.getSymbols()) {
                if (!symbol.getIsBuiltin() &&
!"main".equals(symbol.getSymbolName())) {
                    fwOut.write(scopeLevel + " " + symbol.getSymbolName() + " " +
symbol.getSymbolType() + "\n");
                }
            }
        }
    }
}
```

实际上对于这个总表来说只有一个属性，就是这个Map，将作用域的层数和他对应的`StackSymbolTable`联系起来。所以实际上具体存储符号的逻辑在`StackSymbolTable`中：

```
/* 栈式符号管理当前这一层所拥有的符号 */
public class StackSymbolTable {
    private Map<String, Symbol> SymbolStack = new LinkedHashMap<>();

    public Map<String, Symbol> getSymbolStack() {
        return SymbolStack;
    }
    public Collection<Symbol> getSymbols() {
        return SymbolStack.values();
    }
    public void setSymbolStack(Map<String, Symbol> symbolStack) {
        SymbolStack = symbolStack;
    }
    public void addSymbol(Symbol symbol) {
        getSymbolStack().put(symbol.getSymbolName(), symbol);
    }
    // 判断这个符号是否在这个表中
    public boolean isHaveSymbol(Symbol symbol) {
        Symbol tmp = getSymbolStack().get(symbol.getSymbolName());
        if(tmp == null) {
            return false;
        }
        return true;
    }
```

```
}
```

那这个栈式符号表就更加简单了，只需要一个Map把对应的符号名称和其对应的 Symbol 对应起来就行。这里就是重点说这里对外提供的方法：对于每一层栈式符号表来说，应该能够向这个栈中添加新的符号，因为需要记录这个符号的先后顺序（后续输出需要），这里使用 `LinkedHashMap` ，来记录这个Map。添加和检查是否存在符号的逻辑比较简单。这里为了后续最后输出答案，还写了一个 `printSymbolTable` 方法，可以按照顺序向给定的 `BufferedWriter` 写入符号表的具体内容。

错误的记录

这个错误的记录其实没有什么好说的，就是用一个字符串记录，然后后面的处理方式和语法分析的错误处理是一样的，这里不在过多赘述

```
/* 记录语义分析中出现的错误 */
public class ErrorController {
    private List<ErrorToken> errorTokenList = new ArrayList<>();
    private String errorContent = "";

    public ErrorController() {}

    public ErrorController(String errorContent) {
        this.errorContent = errorContent;
    }

    public void addError(String s) {
        errorContent = errorContent + s;
    }

    public String getErrorContent() {
        return errorContent;
    }
}
```

```
public class ErrorToken {
    private String errorType ;  // 错误码
    private int line ;          // 错误所在行数

    public ErrorToken(String errorType, int line) {
        this.errorType = errorType;
        this.line = line ;
    }
}
```

语义分析

在完成了上述这些准备工作后，就可以正式开始语义分析。按照前面所说这一部分主要完成的有两件事：建立符号表，检测语义错误。

这里要注意的是，这两个工作必须要同时完成，也就是说检测语义错误应该在符号表建立的过程中一起完成。这很容易理解，比如我现在有程序：

```
int a = 1;
b = 2;
int b = 3;
```

如果我把符号表都建立好了，那对于这一层中有符号a 和 b，这个时候 b = 2检测不出来错误。我们需要的是在符号表还没有把 b 放进符号表之前就把这个 b 未定义的错误输出了。

ok，知道了这一点具体我们该怎么做呢？实际上语义分析是一个非常巨大的工程，想要全部完成是非常消耗精力并且复杂的。这里我的建议是一点一点的完成。

1. 对符号的解析

第一步我的建议是完成对符号表的建立，其余错误可以先放在一边。这样做的原因有2点：

1. 建立符号表是必须的，同时也是后续错误检测所必须的基础
2. 建立符号表实际上只需要对声明语法进行处理，工程量较小，适合上手

这里我们只处理一个问题：符号重复定义。其余的后面再做：

观察给定语法，实际上声明一个符号的语法只有这几个：

```
constDecl
    : 'const' bType constDef (',' constDef)* ';'
    ;

constDef
    : IDENT ('[' constExp ']')? '=' constInitVal
    ;

varDecl
    : 'static'? bType varDef (',' varDef)* ';'
    ;

varDef
    : IDENT ('[' constExp ']')? ('=' initVal)?
    ;

funcDef
    : funcType IDENT '(' funcFParams? ')' block
    ;

mainFuncDef
    : 'int' 'main' '(' ')' block
    ;
```

实际上这个 `<constDef>` 只会被这个 `<constDecl>` 调用，`varDef` 也是一样，这个 `mainFunDef` 本质特殊的 `funcDef`，所以实际上我们要完成的只有对 3 个类型语法的解析。

经过之前语法分析的处理，我们得到了一个语法树，我们还有这个语法树的根：`root`，那么做这个语义分析的时候我们就是从这个地方开始。这个地方为了方便进行说明，我这里举一个例子来说明怎么处理，假设源代码如下图所示：

```
int a = 1;
const int b = 2;

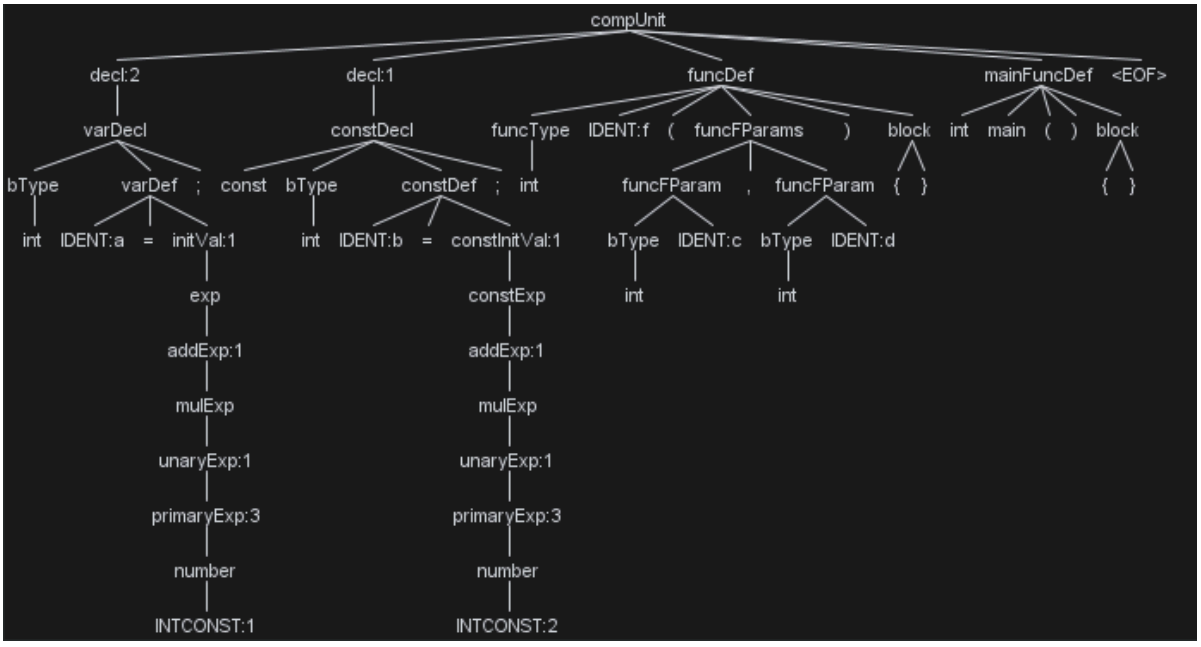
int f(int c, int d) {

}

int main() {

}
```

那么这个时候这个东西对应的语法树就是这个样子的：



首先就是要明确，我们现在有的是这个语法树的根：也就是指向这个 `<compUnit>` 的 root，接下来我们就是从这个 root 开始向下处理。我们首先来处理这个 `<ConstDecl>` ，我们定义一个函数 `traverse` ，用于从当前节点向下处理：

traverse

```
    private void traverse(AstNode node) {
        // traverse 建立参数表
        if(node.isTerminal()) {
            return;
        }
        if(node.getName().equals("<ConstDecl>")) {
            extractConstDecl(node);
            return;
        }
        if(node.getName().equals("<VarDecl>")){
            extractValDecl(node);
            return;
        }
        if(node.getName().equals("<FuncDef>")){
            extractFuncDef(node);
            return;
        }
```

```
        if (node.getName().equals("<Block>")) {
            extractBlock(node);
            return;
        }
        if (node.getName().equals("<MainFuncDef>")) {
            extractMainFuncDef(node);
            return;
        }
        if ("<Exp>".equals(node.getName())
                || "<ConstExp>".equals(node.getName())
                || "<Cond>".equals(node.getName())
                || "<LVal>".equals(node.getName())
                || "<UnaryExp>".equals(node.getName())) {

            analyseExpr(node);
            return;
        }
        if ("<Stmt>".equals(node.getName())) {
            analyseStmt(node);
            return;
        }
        for (AstNode child : node.getChildren()) {
            traverse(child);
        }
    }
```

这里给出的最终的 `traverse`，实际要处理这个 `<constDecl>` 只需要看以下部分：

```
    private void traverse(AstNode node) {
        // traverse 建立参数表
        if(node.isTerminal()) {
            return;
        }
        if(node.getName().equals("<ConstDecl>")) {
            extractConstDecl(node);
            return;
        }
        for (AstNode child : node.getChildren()) {
            traverse(child);
        }
    }
```

因为对于这个常数的定义只会出现在这个 `<constDecl>` 中，我们只需要对这个节点进行特殊处理就可以了。仔细观察上面生成的语法树图，可以发现，这个部分只有下面的几个部分内容：`const`，`type`，`constDef`，`;`，所以按照递归下降的处理方式，这个对应的处理函数就写成：

```
    private void extractConstDecl(AstNode node) {
        boolean isConstant = false;
        String type = null;
        for(AstNode child : node.getChildren()) {
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.CONSTTK) {
                isConstant = true;
            }
```

```
            if (child.isTerminal()
                    && (child.getToken().getType() == TokenType.INTTK
                    || child.getToken().getType() == TokenType.CHARTK)) {
                type = getBType(child);
            }
            if(!child.isTerminal() && child.getName().equals("<ConstDef>")) {
                extractConstDef(type, child, isConstant);
            }
        }
    }
```

通过这种方式来得到对应的const还有类型信息，并把这些信息交给 `extraConstDef` 来进行处理，实际上对这个符号的实际操作在后面这个函数中。这里调用了一个 `getBType` 来从树种得到对应的基础类型：

```
    // 将对应的 BType 的位置转换成对应的类型
    private String getBType(AstNode node) {
        Token token = node.isTerminal()
                ? node.getToken()
                : node.getChildren().get(0).getToken();

        return switch (token.getType()) {
            case INTTK -> "Int";
            case CHARTK -> "Char";
            case VOIDTK -> "Void";
            default -> throw new IllegalStateException("不是类型关键字");
        };
    }
```

这里主要要注意的就是虽然语法中出现了 `<BType>` 这个东西，但是在我们生成树的时候实际上是没有这个 `<BType>` 节点的，所以我们应该像上面的代码写的那样直接看当前是不是叶节点，然后对应的token类型来判断，不能找 `<BType>` 节点。

再观察这个 `<ConstDef>` 的结构，实际上只有 `Ident` ， `=` ， `initVal` ，如果是数组中间还有 `[ <ConstExp> ]` ，现在对这个值的处理先放一边，我们只需要提取符号，建立符号表，并判断是否重复就行了：

```
    private void extractConstDef(String type, AstNode node, boolean isConstant) {
        boolean isArray = false;
        Token token = node.getChildren().get(0).getToken();
        String symbolName = "";
        for(AstNode child : node.getChildren()) {
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.IDENFR) {
                symbolName = child.getToken().getLexeme();
            }
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.LBRACK) {
                isArray = true;
            }
            // 这里是后面的对于初始化的变量类型的处理
            if ("<ConstExp>".equals(child.getName())) {
                traverse(child);
            }
```

```
        }
        SymbolType symbolType = getType(isConstant, false, type, isArray);
        Symbol symbol = new Symbol(symbolName, symbolType, level, scopeId);
        declare(symbol, token);

        // 这里是后面的对于初始化的变量类型的处理
        for (AstNode child : node.getChildren()) {
            if ("<ConstInitVal>".equals(child.getName())) {
                checkInitVal(child, variableType(symbolType));
            }
        }
    }
```

首先得到这个符号的名称，其次就是这个符号是否是一个数组，这里我使用了一个 `getType` 方法来得到这个符号的 `SymbolType`，原因在于题目要求：

语义分析中，不同类型的类型名称统一定义如下：

类型	类型名称	类型	类型名称	类型	类型名称
int型变量	Int	int型静态变量数组	StaticIntArray	char型常量数组	ConstCharArray
int型常量	ConstInt	char型变量	Char	char型静态变量数组	StaticCharArray
int型静态变量	StaticInt	char型常量	ConstChar	void型函数	VoidFunc
int型变量数组	IntArray	char型静态变量	StaticChar	int型函数	IntFunc
int型常量数组	ConstIntArray	int型变量数组	CharArray	char型函数	CharFunc

这个也可以说是非常猎奇了，为了满足这个的要求，我就写了一个函数，来根据这个符号的属性生成对应的类型：

```
    // 获得语义分析对应的变量
    private SymbolType getType(boolean isConstant, boolean isStatic, String
btype, boolean isArray) {
        String typeName = "";
        if(isConstant) typeName = "Const";
        else if(isStatic) typeName = "Static";

        typeName += btype;

        if(isArray) typeName += "Array";
        return SymbolTypeMap.getSymbolTypeMap().get(typeName);
    }
```

而这个 `declare` 就是用来向符号表写入这个符号的方法：

```
    public void report(Token token, String code) {
        errorController.addError(token.getLine() + " " + code + "\n");
    }

    // 处理 b 类错误 ： 重命名
    private void declare(Symbol symbol, Token token) {
        if(!symbolTable.addSymbol(symbol)) {
            report(token, "b");
        }
    }
```

尝试向符号表中写入符号，如果能写入直接结束，否则就会报告一个 b 类错误。

同时这里还要注意这个对 `level 和 scopeID` 的处理。这个地方还是有一些问题的，这里就直接说明本处的设计，这里我也是参考了 ai 的建议，这个level是我本来自己设计的，用来标志这个层次，但是我的设计有很大的问题，在这个地方可以完全不管了。这个 `scopeID` 用来完成对这个符号所在的作用域进行管理，这里使用的也是一种栈式的方式，使用一个栈来保存层次，进入一个新的层次的时候就把当前所在的层次号入栈，并且把新的层次变为 `nextScopeID + 1`， 在完成了当前层次的内容之后，再弹出栈顶的元素，赋值给 `scopeID`。通过这样的方式实现了对符号进行分层的管理。

```
private int scopeId = 1; // 当前所在作用域
private int nextScopeId = 1;    // 下一个进入的作用域
private Deque<Integer> scopeStack = new ArrayDeque<>();
private void levelUp() {
    // 保存父作用域编号
    scopeStack.push(scopeId);
    // 创建新的作用域编号
    scopeId = ++nextScopeId;
}

private void levelDown() {
    // 恢复父作用域编号
    scopeId = scopeStack.pop();
}
```

这个地方其实关键在于 `scopeID` 更新的时候是根据这个 `nextScopeID + 1` 来进行的，不能使用 `scopeID + 1` 来进行。后者的问题在于，比如现在有一个源代码：

```
{ // a
    int a = 1;
    { // b
        int b = 1;
    }
}

{ // c
    int b = 1;
}
```

如果按照 `scopeID + 1` 来进行更新的话就会出现 b 和 c 两处属于同一个 `scopeID`，这就会被认为这个 b 重复定义了，但是很显然这样是不正确的。按照 `nextScopeID + 1` 来更新就避免了这个问题。

这样就完成了对于这个 `<ConstDecl>` 的定义的符号的管理了，成功的把这个定义的符号放进了保存进入了这个符号表中，同时在放入符号表的时候还检测了是否存在重复定义的问题。那么接下来这个 `<ValDecl>` 和这个非常类似，这里就不再过多重复：

```
    // 对变量声明的处理
    private void extractValDecl(AstNode node) {
        boolean isStatic = false;
        String type = null;
        for(AstNode child : node.getChildren()) {
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.STATICTK) {
                isStatic = true;
```

```
            }
            if (child.isTerminal()
                    && (child.getToken().getType() == TokenType.INTTK
                    || child.getToken().getType() == TokenType.CHARTK)) {
                type = getBType(child);
            }
            if(!child.isTerminal() && child.getName().equals("<VarDef>")) {
                extractValDef(isStatic, type, child);
            }
        }
    }

    private void extractValDef(boolean isStatic, String type, AstNode node) {
        boolean isArray = false;
        Token token = node.getChildren().get(0).getToken();
        String symbolName = "";
        for(AstNode child : node.getChildren()) {
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.IDENFR) {
                symbolName = child.getToken().getLexeme();
            }
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.LBRACK) {
                isArray = true;
            }
            // 这里是后面的对于初始化的变量类型的处理
            if ("<ConstExp>".equals(child.getName())) {
                traverse(child);
            }
        }
        SymbolType symbolType = getType(false, isStatic, type, isArray);
        Symbol symbol = new Symbol(symbolName, symbolType, level, scopeId);
        declare(symbol, token);

        // 这里是后面的对于初始化的变量类型的处理
        // 登记当前变量后，再检查初始化表达式
        for (AstNode child : node.getChildren()) {
            if ("<InitVal>".equals(child.getName())) {
                checkInitVal(child, variableType(symbolType));
            }
        }
    }
```

还剩下最后一个函数的声明，这里的特殊点在于和普通的符号不一样，一个函数还有他对应的返回值和参数列表。但是本质上处理思路还是和上述的变量声明一致。还是先观察这个树的结构，按照这个树的结构一样写：

```
    private void extractFuncDef(AstNode node) {
        // 这个地方要注意参数的插入顺序：先函数，后函数的参数，最后是block中的内容，顺序不能错
误
        String type = null;
        String symbolName = "";
        AstNode paramsNode = null;
        AstNode blockNode = null;
        Token token = null;
```

```
        for (AstNode child : node.getChildren()) {
            if (!child.isTerminal() && child.getName().equals("<FuncType>")) {
                type = getBType(child);
            }
            if (child.isTerminal()
                    && child.getToken().getType() == TokenType.IDENFR) {
                symbolName = child.getToken().getLexeme();
                token = child.getToken();
            }
            if (!child.isTerminal() && child.getName().equals("<FuncFParams>")) {
                paramsNode = child;
            }
            if (!child.isTerminal() && child.getName().equals("<Block>")) {
                blockNode = child;
            }
        }
        // 2. 仅提取形参类型，不插入形参符号
        List<SymbolType> paramTypes = new ArrayList<>();
        if (paramsNode != null) {
            paramTypes = collectParamTypes(paramsNode);
        }
        // 3. 注册函数符号
        SymbolType symbolType = getFuncType(type);
        FuncSymbol funcSymbol = new FuncSymbol(
                symbolName, symbolType, level, scopeId, paramTypes,
variableType(getFuncType(type))
        );
        declare(funcSymbol, token);

        // 4. 进入函数作用域
        ExprType previousType = currentExprType;
        currentExprType = variableType(getFuncType(type));
        levelUp();
        scopeId = nextScopeId;
        // 5. 注册形参
        if (paramsNode != null) {
            extractFuncFParams(paramsNode);
        }
        // 6. 遍历函数体子节点
        if (blockNode != null) {
            for (AstNode child : blockNode.getChildren()) {
                traverse(child);
            }
        }
        if(currentExprType == ExprType.CHAR || currentExprType == ExprType.INT) {
            checkMissingReturn(blockNode);
        }
        // 7. 退出函数作用域
        levelDown();
        currentExprType = previousType;
    }
```

就像注释写的一样，这个地方要尤其注意符号表的处理顺序，首先要定义函数符号，然后是函数的参数，最后才是函数的block中的内容，这个顺序是不能改的。

那么在定义这个函数符号的时候，需要的只有参数的类型即可，用于后续判断调用函数的时候参数列表的类型是否正确，而后面对函数参数符号进行定义的时候不仅需要参数的类型，还需要参数的具体名称，这个用于在函数内容调用是使用。所以这里使用了两个不同的方法分别对 `<ParamsNode>` 进行解析： `collectParamTypes` 和 `extractFuncFParams`。

```
    private List<SymbolType> collectParamTypes(AstNode node) {
        List<SymbolType> paramTypes = new ArrayList<>();
        for (AstNode child : node.getChildren()) {
            if (child.isTerminal() || !"<FuncFParam>".equals(child.getName())) {
                continue;
            }
            String btype = null;
            boolean isArray = false;
            for (AstNode paramChild : child.getChildren()) {
                if (paramChild.isTerminal()
                        && (paramChild.getToken().getType() == TokenType.INTTK
                        || paramChild.getToken().getType() == TokenType.CHARTK))
{
                    btype = getBType(paramChild);
                }
                if (paramChild.isTerminal()
                        && paramChild.getToken().getType()
                        == TokenType.LBRACK) {
                    isArray = true;
                }
            }
            SymbolType type = getType(false, false, btype, isArray);
            paramTypes.add(type);
        }
        return paramTypes;
    }

        private List<SymbolType> extractFuncFParams(AstNode node) {
        List<SymbolType> paramTypes = new ArrayList<>();
        for(AstNode child : node.getChildren()) {
            if(!child.isTerminal() && child.getName().equals("<FuncFParam>")) {
                paramTypes.add(extractFuncFParam(child));
            }
        }
        return paramTypes;
    }

    private SymbolType extractFuncFParam(AstNode node) {
        String btype = null;
        boolean isArray = false;
        String symbolName = "";
        Token token = null;
        for(AstNode child : node.getChildren()) {
            if (child.isTerminal()
                    && (child.getToken().getType() == TokenType.INTTK
                    || child.getToken().getType() == TokenType.CHARTK)) {
                btype = getBType(child);
            }
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.LBRACK) {
```

```
                isArray = true;
            }
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.IDENFR) {
                symbolName = child.getToken().getLexeme();
                token = child.getToken();
            }
        }
        SymbolType symbolType = getType(false, false, btype, isArray);
        Symbol symbol = new Symbol(symbolName, symbolType, level, scopeId);
        declare(symbol, token);
        return symbolType;
    }
```

其实具体实现的思路还是一样的，这里不再赘述。

实现了这个对一般函数的处理，那么是现在对这个特殊的 main 函数的处理就更加简单了：

```
    private void extractMainFuncDef(AstNode node) {
        String symbolName = "main";
        Token token = null;
        AstNode blockNode = null;
        SymbolType symbolType = getFuncType("Int");
        for(AstNode child : node.getChildren()) {
            if(child.isTerminal() && child.getToken().getType() ==
TokenType.MAINTK) {
                token = child.getToken();
            }
            if("<Block>".equals(child.getName())) {
                blockNode = child;
            }
        }
        List<SymbolType> paramTypes = new ArrayList<>();
        Symbol symbol = new FuncSymbol(symbolName, symbolType, level, scopeId,
paramTypes, ExprType.INT);
        declare(symbol, token);

        ExprType previousType = currentExprType;
        currentExprType = ExprType.INT;

        for(AstNode child : node.getChildren()) {
            if(!child.isTerminal()) {
                traverse(child);
            }
        }

        //后续的错误处理
        checkMissingReturn(blockNode);
        currentExprType = previousType;
    }
```

在完成了上面的操作之后，实际上这个符号表就已经被我们建立好了。这也就意味着我们已经完成了语义分析的第一个工作： 建立符号表。但是这还远远没有结束，除了建立符号表，我们还需要对各种各样的错误进行检测，这也就是我们后续需要进一步完成的事情。

错误检测

首先观察给定的错误类型列表，除了在词法分析和语法分析中会出现的错误外，剩下的在语义分析中可能出现的错误中，除了错误类型 b（名字重定义）和 f（无返回值的函数有不匹配的return），g（有返回值的函数没有return） 这几种错误会出现在符号的定义过程，其余错误几乎就都是在使用符号的过程中才会出现。而这些错误大多数都和类型有关。这里的类型其实就是前面定义的这个枚举类 `ExprType` 。在后续的错误检测中，这个类型会非常普遍的被使用。这里的难点在于这里会出现大量的表达式相关的内容。比如源代码中可能会出现这样的内容：

```
int a = 1;
int b = a + a * a - 1 ;
```

可以看到这个 b 的初值不是一个常量，而是一个表达式，这个时候进行语义分析的时候，为了判断 b 定义的类型和后面的表达式的类型是否一致时，就无法避免的需要对后面这个表达式进行处理。而这种类似的情况非常普遍的出现在各种位置。所以在进行后续的语义错误检测的时候，我们应该首先完成这个对表达式的类型的解析工作。需要实现的效果就是对于这种表达式，一方面能够找到表达式自己在计算的时候类型是否正确对应，另一方面就是这个表达式运行得到的最终类型。

表达式处理

首先看看语法中和表达式中所有相关的内容

```
exp
    : addExp
    ;

cond
    : lOrExp
    ;

lVal
    : IDENT ('[' exp ']')?
    ;

primaryExp
    : '(' exp ')'
    | lVal
    | number
    ;

number
    : INTCONST
    | CHARCONST
    ;

unaryExp
    : primaryExp
    | IDENT '(' funcRParams? ')'
    | unaryOp unaryExp
    | '(' bType ')' unaryExp
    ;
```

```
unaryOp
    : '+'
    | '-'
    | '!'
    ;

funcRParams
    : exp (',' exp)*
    ;

mulExp
    : unaryExp (('*' | '/' | '%') unaryExp)*
    ;

addExp
    :  mulExp | addExp ('+' | '-') mulExp
    ;

relExp
    : addExp (('<' | '>' | '<=' | '>=') addExp)*
    ;

eqExp
    : relExp (('==' | '!=') relExp)*
    ;

lAndExp
    : eqExp ('&&' eqExp)*
    ;

lOrExp
    : lAndExp ('||' lAndExp)*
    ;

constExp
    : addExp
    ;
```

仔细观察文法（或者使用 gpt），可以对表达式进行如下分类

1. `Exp`、`ConstExp`、`Cond`：直接向下传递，在当前文法中，只有一个用于承载表达式的子节点，自己不引入新的运算。
2. `Number`：直接确定类型，可以通过对 `Number` 下面的 token 进行分析得到对应的类型
3. `PrimaryExp`：根据产生式选择子节点，并有多种选择
4. `LVal`：使用一个符号，包括普通变量和数组
5. `PrimaryExp`：有多种选择
6. `AddExp` 和 `MulExp`： 普通计算的表达式
7. `LAndExp`，`LOrExp`，`EqExp`，`RelExp`： 逻辑计算的表达式

这里 6 和 7 的区别在于指导书中有要求：

3. `char` 类型变量可以单独使用，也可以进行 `char` 类型之间的算术运算，（算术）运算结果仍为 `char` 类型。`char` 类型参与条件运算（`==`、`!=`、`<`、`>`、`<=`、`>=`）或逻辑运算（`&&`、`||`），结果为 `int`。
4. `int` 类型变量可以单独使用，也可以进行 `int` 类型之间的算术运算，（算术）运算结果仍为 `int` 类型。`int` 类型参与条件运算（`==`、`!=`、`<`、`>`、`<=`、`>=`）或逻辑运算（`&&`、`||`），结果为 `int`。

也就是说，6中的两个 char 运行的结果还是char，而通过 7 处理应该输出 int，这也是为什么将两者进行拆分。

按照这个分组，可以初步的写出对表达式解析的方法：

```
    // 对表达式进行处理，找出表达式内部的问题，同时返回表达式的类型
    private ExprType analyseExpr(AstNode node) {
        return switch (node.getName()) {
            case "<Exp>", "<ConstExp>", "<Cond>" ->
                    analyseExpr(node.getChildren().get(0));

            case "<Number>" -> {
                Token token = node.getChildren().get(0).getToken();
                yield token.getType() == TokenType.INTCON
                        ? ExprType.INT
                        : ExprType.CHAR;
            }
            case "<PrimaryExp>" -> {
                AstNode first = node.getChildren().get(0);

                // '(' Exp ')'：分析中间的 Exp
                if (first.isTerminal()) {
                    yield analyseExpr(node.getChildren().get(1));
                }

                // LVal 或 Number
                yield analyseExpr(first);
            }

            case "<LVal>" -> analyseLVal(node);

            case "<MulExp>", "<AddExp>" ->
                    analyseArithmeticExpr(node);

            case "<UnaryExp>" -> analyseUnaryExpr(node);

            case "<RelExp>", "<EqExp>",
                 "<LAndExp>", "<LOrExp>" ->
                    analyseConditionExpr(node);

            default -> throw new IllegalStateException(
                    "尚未处理的表达式节点：" + node.getName()
            );
        };
    }
```

对于不同分组的表达式有不同的处理方法。其中 `<Number>` 最简单，只需要直接找其对应的 token 的类型就行了，直接返回对应的类型。

其中比较关键的是对 `<MulExp>` 和 `<AddExp>` 的处理，其他的几乎都是一样的思想，只能说是大同小异，这里具体说说这个 `analyseArithmeticExpr`：

```
    private ExprType analyseArithmeticExpr(AstNode node) {
        List<AstNode> children = node.getChildren();

        // 没有实际运算，直接传递孩子的类型
        if (children.size() == 1) {
            return analyseExpr(children.get(0));
        }
        // 先分析两边，保证右边的错误也能被发现
        ExprType left = analyseExpr(children.get(0));
        ExprType right = analyseExpr(children.get(2));

        Token operator = children.get(1).getToken();
        // 子表达式已经出错，不再由当前运算追加类型错误
        if (left == ExprType.ERROR || right == ExprType.ERROR) {
            return ExprType.ERROR;
        }
        boolean sameType = left == right;
        boolean basicType = left == ExprType.INT
                || left == ExprType.CHAR;

        if (!sameType || !basicType) {
            report(operator, "e");
            return ExprType.ERROR;
        }
        // INT 运算得到 INT，CHAR 运算得到 CHAR
        return left;
    }
```

这里要注意这个 `ExprType.ERROR` 这个类型，这个其实很关键，举一个例子：

```
int a = 1;
char b = 2;
int c = a + b + 3;
```

在处理 c 这个表达式的时候，a 和 b 的类型不一样，那么就会在处理 a + b这个时候输出一个错误，那之后呢，继续往后面 处理到 + 3 的时候，会不会继续输出错误呢？实际上 a + b 都是错误的了，剩下的处理其实没有什么用了，但是实验的要求是在出现错误的时候还要继续往下进行分析错误，所以这里使用一个 ERROR 类型来表示 a + b 的结果，这样就在后续处理的过程中，如果处理的部分中已经出现了 ERROR，那就不应该继续输出错误了，因为这个已经在前面输出了错误，而且说白了，这里你也无法输出错误。

这个部分的内容非常丰富，但其实总体的思路是一样的，都是根据这个语法一步一步解析其中的类型，如果出现问题通过report报告这个问题。个人感觉这个地方应该是一个非常困难的地方了，本人写的时候在这个地方思考了很久，最终在 gpt 的帮助下完成，代码也是 gpt 生成的居多。本人对这段的理解也比较浅显，这里就不过多不懂装懂了。

错误处理

在完成了上面的这个表达式处理之后，就可以正式的开始对这个具体错误的检测了。我这里的具体写法是按照给定的错误表格一个一个去写检测错误的代码。这一块的内容也是非常之多啊，我也不全部说明，这里选择其中几个说明。

当然我们应该意识到，正如前面所说，这个语义错误除了前面提到的那几个，剩下的几乎全部都是在使用一个变量出现的。那很显然，我们的代码只有在一个 block 内才会使用变量，所以实际上，这些处理都是在 block 的内部来进行的。这个的意思是，除了在 block 外面可能出现的表达式，赋值，定义之外，其余的都在 block 内部，所以我们总体的 traverse 完全可以写成之前的那种形式 traverse ,只需要处理这几种类型完全足够。

这里重点说明这个 e 类型的错误：

类型不匹配	e	当表达式中出现多类型(指 `int/char/int[]/char[]/void` 中任意类型)混合运算时，或者有返回值的函数返回的类型与定义的返回类型不一致，或者赋值语句中左值和右值类型不一致，或者函数参数传递时实参类型与形参类型不一致，或者 `switch` 语句中 `case` 的 `Number` 类型与 `switch` 表达式类型不一致，或者 `printf` 语句中格式字符串与表达式类型不一致时,或者函数标识符与变量相互误用时，报此错误。报错行号为**发生类型不匹配的位置**所在行号。	`Stmt → LVal '=' Exp ';'` `Stmt → 'return' [Exp] ';'` `UnaryExp → Ident '(' [FuncRParams] ')'` `CaseStmt → 'case' Number ':' ...` `Stmt → 'printf''('StringConst {','Exp}')'';'`

我认为这个应该是最复杂的一个错误了。但是实际上完成的时候就完全看最右边这一栏的内容，根据给出的可能出现错误的语法进行定义，通过这样的方式一个一个进行修改就可以了。这里选择其中的第一个 `stmt -> LVal '=' Exp ';'` 这个来具体说说。

首先这个是在对 stmt 的处理中所以有：

```
    // 处理 stmt 中可能出现的错误
    private void analyseStmt(AstNode node) {
        List<AstNode> children = node.getChildren();
        AstNode first = children.get(0);
        if("<LVal>".equals(children.get(0).getName())) {
            checkAssignment(node);
            return ;
        }
        if (first.isTerminal()) {
            TokenType type = first.getToken().getType();
            switch (type) {
                case RETURNTK -> {
                    checkReturn(node);
                    return;
                }
                case SWITCHTK -> {
                    switchDepth++;
                    try {
                        checkSwitch(node);
                    } finally {
                        switchDepth--;
```

```
                    }
                    return;
                }
                case PRINTFTK -> {
                    checkPrintf(node);
                    return;
                }
                case WHILETK -> {
                    loopDepth++;
                    try {
                        for(var child : node.getChildren()) {
                            traverse(child);
                        }
                    } finally {
                        loopDepth--;
                    }
                    return ;
                }
                case BREAKTK -> {
                    // break 必须处于 while 或 switch 中
                    if (loopDepth == 0 && switchDepth == 0) {
                        report(first.getToken(), "m");
                    }
                    return;
                }
                case CONTINUETK -> {
                    // continue 必须处于 while 中
                    if (loopDepth == 0) {
                        report(first.getToken(), "m");
                    }
                    return;
                }
            }
        }
        for (AstNode child : children) {
            traverse(child);
        }
    }
```

通过这个函数将具体的处理分发到 `checkAssignment` 中完成

```
    // 处理 stmt -> lVal = Exp ; 这个语法错误
    private void checkAssignment(AstNode node) {
        AstNode lValNode = null;
        AstNode expNode = null;
        for (AstNode child : node.getChildren()) {
            if ("<LVal>".equals(child.getName())) {
                lValNode = child;
            } else if ("<Exp>".equals(child.getName())) {
                expNode = child;
            }
        }
        if (lValNode == null || expNode == null) {
            return;
        }
```

```
        Token lValToken = lValNode.getChildren().get(0).getToken();
        Symbol lValSymbol = findSymbol(lValToken.getLexeme());

        if(lValSymbol != null && isConstant(lValSymbol)) {
           report(lValToken, "h");
        }

        ExprType left = analyseExpr(lValNode);
        ExprType right = analyseExpr(expNode);
        if (left == ExprType.ERROR || right == ExprType.ERROR) {
            return;
        }
        if (left != right) {
            Token token = lValNode.getChildren().get(0).getToken();
            report(token, "e");
        }
    }
```

依旧是根据语法一点一点写，然后使用之前的 `analyseExpr` 方法轻松分析表达式类型，巴拉巴拉比较得到结果。当然在这个内部后续也可以根据需求进一步填充其他的错误检测，比如这里的

```
        if(lValSymbol != null && isConstant(lValSymbol)) {
           report(lValToken, "h");
        }
```

就是后面的 h 错误的检测。其余的只能说大同小异，总的处理思想是这样的。但是不管怎么说想要总体完成这个任务再急头白脸的de一下bug，那可以说是非常享受了。这里我就不过多说明，具体可以看代码。

按照这样的方式进行下去，就完成了错误检测这一部分。

完善代码以通过测试

写到这个地方，这个语义分析就几乎是写完了。但是这样的话实际上是过不了测试的。这是因为有几个问题：

1. 本次作业需要建立符号表，**除了要求的输出内容，其余符号表信息请大家自行设计，但是额外的信息请不要输出**。
2. **设计符号表请考虑后续中间代码生成过程中对符号表的使用**。
3. **不应输出libsysy 运行时库中的符号（例如，get_int、get_char）**。
4. 当前要求的正确源程序的输出只是为了便于评测，完整的编译器最后无需出现这些信息，请设计为方便打开/关闭这些输出的方案。
5. 本次作业中对于每个错误样例，可能出现所有类型的错误。
6. **关于作用域序号，即进入该作用域之前进入的作用域数量加1**。进入全局作用域时进入的作用域数量为0，因此全局作用域序号为1。
7. 关于符号表的输出，按照作用域序号大小依次输出，先输出序号较小的作用域中的全部符号。对于同一个作用域，按照符号被声明的先后顺序输出。
8. **函数的参数属于函数内部的作用域**。
9. `main` 是保留的关键字，不纳入符号表中。

注意 2 和 9，9 的意思很明显，就是 main 这个符号不能放入符号表。

2 的意思是，对于程序中可能出现的 `get_int` 这一类函数的时候，我们一方面不能把他识别成未定义的符号，另一方面，这个符号不能被最后输出。那我这边的处理方式是给符号加一个属性：`private boolean isBuiltin = false;` 然后在建立符号表的之前先进行一个预处理：

```
    private void initBuiltinFunctions() {
        List<SymbolType>list = new ArrayList<>();
```

```
        Symbol symbol = new FuncSymbol("get_int", SymbolType.IntFunc, 1, 1, list,
ExprType.INT);
        symbol.setIsBuiltin(true);
        symbolTable.addSymbol(symbol);

        symbol = new FuncSymbol("get_char", SymbolType.CharFunc, 1, 1, list,
ExprType.CHAR);
        symbol.setIsBuiltin(true);
        symbolTable.addSymbol(symbol);

        String[] strings = {"get_string", "put_int", "put_char", "put_string"};
        SymbolType[][] symbolLists = {
                {
                    SymbolType.CharArray, SymbolType.Int
                },
                {
                    SymbolType.Int
                },
                {
                    SymbolType.Char
                },
                {
                    SymbolType.CharArray
                },
        };

        for(int i = 0; i < strings.length; i++) {
            symbol = new FuncSymbol(strings[i], SymbolType.VoidFunc, 1, 1,
Arrays.stream(symbolLists[i]).toList(), ExprType.VOID);
            symbol.setIsBuiltin(true);
            symbolTable.addSymbol(symbol);
        }
    }
```

对于这些符号，再放入符号表的时候把他的 `isBuiltin` 属性设置成 true ，然后在最后的输出中增加一个判断，必须是false才输出，这样就很好的解决了这个问题。

end

进行到这个地方，最后再把 main 稍微改改，把这个语义分析器放进 main 中从根节点开始分析，再改改文件名什么玩意的就结束了。

总的来说感觉这个语义分析还是比较难的，而且工作量也很大。